

1 Criando o projeto no MPLAB X

Todo o firmware desta disciplina nasce de um projeto igual a este. Faça o percurso uma vez com atenção: nos roteiros seguintes ele será resumido a uma linha.

CONCEITO

O MPLAB X é usado aqui **apenas como compilador**. Ele não grava o microcontrolador: quem faz isso é o aplicativo do bootloader, que roda separado. Por isso o projeto é criado no modo **No Tool** — ausência de gravador é a configuração correta, não um defeito.

1.1 Passo a passo

1. **File** → **New Project** (ou Ctrl+Shift+N).
2. Categoria **Microchip Embedded**, tipo **Standalone Project**. Avançar.
3. Em **Device**, digite PIC18F4550. Confira que a família selecionada é **Advanced 8-bit MCUs (PIC18)**. Avançar.
4. Em **Select Tool**, escolha **No Tool**. Avançar.
5. Em **Select Compiler**, escolha o **XC8** disponível na máquina. Avançar.
6. Em **Project Name**, use termostato. Confira a pasta de destino e conclua em **Finish**.
7. No painel **Projects**, clique com o botão direito em **Source Files** → **New** → **C Main File**. Nomeie main.c e conclua.
8. Apague o conteúdo gerado automaticamente e escreva o programa da seção seguinte.
9. Compile com o ícone de martelo (**Build Project**, F11). A janela **Output** deve terminar com BUILD SUCCESSFUL.

ATENÇÃO

O caminho da pasta do projeto não pode conter **acentos, espaços ou cedilha**. Um caminho como C:\Users\João\Meus Projetos\ faz o XC8 falhar com mensagens que não têm relação aparente com a causa. Use algo como C:\micro\termostato.

1.2 Onde fica o arquivo gravável

A compilação não produz um arquivo na pasta do projeto: ela produz um .hex dentro da árvore de saída. Localize-o antes de abrir o bootloader.

NA BANCADA

<pasta do projeto>\dist\default\production\termostato.production.hex

TAREFA

0.1 — Anote o caminho completo do .hex do seu projeto. Você vai procurá-lo em todas as sessões seguintes.

1.3 Gravação pelo bootloader

1. Abra o aplicativo **Bootloader PIC18/XM118** (v2.8). Não use o MPLAB X para gravar.
2. Pressione **SW9** (RESET) no kit. A partir daí você tem uma janela de aproximadamente **4 a 5 segundos** para o aplicativo reconhecer a placa.
3. Com o dispositivo conectado, abra o .hex localizado acima e execute a gravação.
4. Pressione **SW9** novamente para sair do bootloader e executar o programa.

DIVERGÊNCIA DE DOCUMENTAÇÃO

O manual da Exsto indica **SW1** como botão de reset. Na placa, a serigrafia identifica **SW9**. Quando manual e serigrafia divergem, **a serigrafia vence**.

NOTA

Se a janela de 4 segundos passar, o programa antigo volta a rodar e o aplicativo não encontra a placa. Não é falha de cabo nem de driver: basta pressionar SW9 de novo.

2 O programa de referência

CONCEITO

A ordem LAT antes de TRIS é a regra de segurança central deste roteiro: primeiro se decide **qual valor o pino terá**, depois se autoriza o pino a **sair para o mundo**. Invertida a ordem, o pino passa por um instante em estado indefinido — irrelevante para um LED, inaceitável para um aquecedor.

```
#include <xc.h>
#define _XTAL_FREQ 16000000UL

void main(void) {
    LATD = 0xFF;    // valor primeiro
    TRISD = 0x00;    // direção depois

    while (1) {
        LATDbits.LATD0 = 1;
        __delay_ms(500);
        LATDbits.LATD0 = 0;
        __delay_ms(500);
    }
}
```

DIVERGÊNCIA DE DOCUMENTAÇÃO

Diretivas #pragma config escritas na aplicação são **silenciosamente ignoradas** neste kit: os bits de configuração pertencem ao bootloader, que já os fixou. O bootloader configura 48 MHz para o periférico USB e **16 MHz para a CPU**. Por isso _XTAL_FREQ vale 16000000UL, e não 48000000UL.

3 Do delay até a tela do osciloscópio

O compilador precisa de _XTAL_FREQ para converter um atraso em milissegundos num número de ciclos de máquina. Com $F_{osc} = 16 \text{ MHz}$:

$$T_{cy} = \frac{4}{F_{osc}} = \frac{4}{16 \times 10^6} = 250 \text{ ns}$$

O laço acima acende por 500 ms e apaga por 500 ms. O período da onda é a soma das duas metades, e a frequência é o seu inverso.

3.1 Previsão

EXPERIMENTO

Antes de ligar o osciloscópio, preencha a coluna de frequência prevista. Some as duas metades do laço para obter o período e inverta-o. Rubrique a tabela e mostre ao professor. A nota é pelo raciocínio registrado, não pelo acerto.

<code>_delay_ms</code> (cada metade)	Período previsto	f prevista	f medida	Erro (%)
500				
250				
100				
50				

Tabela 1: Previsão e medição da onda quadrada em RD0.

3.2 Medição

EXPERIMENTO

Conecte a ponta de prova ao pino do LED que pisca e a garra de terra a um ponto de **GND** do kit. Meça o período com os cursores e preencha as duas últimas colunas da tabela. Recompile e regrave a cada mudança de atraso.

TAREFA

0.2 — O erro percentual é aproximadamente o mesmo nas quatro linhas ou cresce com a frequência? O que isso diz sobre a origem do erro: uma fonte proporcional (o clock) ou uma fonte fixa (as instruções do laço que não são o atraso)?

3.3 Descobrindo o clock pela medida

Suponha que um colega tenha declarado `_XTAL_FREQ` como 48000000UL, por ter lido no manual que o kit opera a 48 MHz. O compilador acredita na declaração e calcula os ciclos do atraso a partir dela.

TAREFA

0.3 — Se o clock real da CPU é 16 MHz e o compilador supôs 48 MHz, o atraso executado será mais **curto** ou mais **longo** que o pedido, e por qual fator? Deduza antes de testar.

0.4 — Troque `_XTAL_FREQ` para 48000000UL, recompile, grave e meça o novo período. A razão entre o período medido e o previsto confirma sua dedução?

0.5 — Inverta o raciocínio: partindo apenas de um período medido no osciloscópio e do valor declarado em `_XTAL_FREQ`, escreva a expressão que recupera a frequência real da CPU. Este é o método que você usará sempre que suspeitar do clock.

NOTA

Guarde este resultado. Toda temporização do semestre — o Timer0 do termostato, o baud rate da comunicação serial, a frequência do PWM — depende de `_XTAL_FREQ` estar correto. Um clock supostamente errado é a primeira hipótese a testar quando um periférico temporizado se comporta de forma estranha.

4 O limite do olho

TAREFA

0.6 — Reduza o atraso progressivamente e registre a partir de qual valor o LED **parece continuamente aceso**, sem piscar perceptível. Converta esse atraso em frequência.

0.7 — Nessa condição, o LED ainda está piscando? O que o osciloscópio mostra? Explique por que, acima dessa frequência, o osciloscópio deixa de ser um instrumento conveniente e passa a ser o **único** capaz de revelar o que o pino está fazendo.

CONCEITO

Você acaba de encontrar experimentalmente o princípio do controle por largura de pulso: acima de algumas dezenas de hertz, a carga — o olho, um motor, um aquecedor — responde à **média** do sinal, não a cada transição. No Roteiro 6 esse mesmo fenômeno reaparece a 977 Hz, com o hardware do CCP1 gerando a onda no lugar do laço de atraso.